

Welcome back to CS 2100!

Prof. Rasika Bhalerao

Poll: How can I swap the values of `x: int` and `y: int`?

a.

```
temp: int = x
x = y
y = temp
```

c.

```
temp: int = y
x = y
x = temp
```

b.

```
temp: int = x
temp = y
y = x
```

d.

```
temp: int = x
y = temp
x = y
```

Poll: What does this print? `print("I am so excited" + "!" * 3)`

1. I am so excited! I am so excited! I am so excited!
2. I am so excited!I am so excited!I am so excited!
3. I am so excited!!!
4. Nothing – it breaks

Fun with Python: `str` times `int`

We use quotes to represent strings.

Single quotes `'hi'` or double quotes `"hi"`

So... How can we put quotes in a string?

```
"Easiest way to put an apostrophe ' is to use double quotes '"  
'Easiest way to put quotes " is to use single quotes '"
```

What if I want both `'` and `"` in the string?

[Python documentation that may help](#)

Thanks, but that was very ugly.

F-string

Preferred solution: put the variable directly in the string with { brackets }

```
cats: int = 4  
print(f"There are {cats} cats in this room.")
```

```
There are 4 cats in this room.
```

Cleaner, reduces opportunities for bugs, impresses your boss...

Let's practice writing tests for this function:

```
def get_cat_phrase(num: int) -> str:
    """Return a string describing the number of cats, formatted
    like: '<num> cats' (where the 's' is only added if the number
    of cats is not 1)
```

Args:

```
    num : int
        The number of cats
```

Returns:

```
    str : A string representing the number of cats
```

Raises:

```
    ValueError if the number of cats is negative
    """
```

```
    if num < 0:
        raise ValueError(f'Cannot have {num} cats')
    else:
        return f'{num} cat{"s" if num != 1}'
```

Float

Normal activity: run this:

```
print(5 / 2)
print(type(5 / 2))
```

```
2.5
<class 'float'>
```

Fun activity: run this:

```
print(4 / 2)
print(type(4 / 2))
```

Formatting floats

Let's figure out how to print `price` with exactly two decimal places.

Starting point:

```
price = 12.345678
print(f'{price}')
```

Evaluate:

`7 < 2 + 8`

Evaluate:

`3 < 4 and 5 < 7`

Order of operations

- Math happens before comparison operations
- Comparison happens before boolean operations

Mutation testing: how we autograde your tests

"Mutation testing": programmers insert small, common bugs into their code and check whether the tests catch them.

We grade student tests by checking that they:

1. Pass on correct code
2. Fail on incorrect code

(1) is needed before moving on to (2)

Disclaimer: All bugs that we inserted into the incorrect code are intended to be simple and common. If the autograder says there is a bug that remains undetected by your tests, look for large missing test cases, rather than digging into obscure ways code can run incorrectly.

```
def add(a: int, b: int) -> int:
    """Returns the sum of two integers."""
    return a + b

def test_add_positive_numbers() -> None:
    """Test adding two positive numbers."""
    assert False

def test_add_negative_numbers() -> None:
    """Test adding two negative numbers."""
    assert add(-1, -1) == -2

def test_add_mixed_numbers() -> None:
    """Test adding a positive and a
    negative number."""
    assert add(-1, 1) == 0

def test_add_zero() -> None:
    """Test adding zero to a number."""
    assert add(0, 5) == 5
    assert add(5, 0) == 5
```

Poll: Why is this assignment submission not receiving full points?

1. The student implemented `add()` incorrectly.
2. Mutation testing: the student's tests don't cover enough cases.
3. The student's test fails on correct code, so mutation tests are not run.
4. It's something else -- pylint warnings, infinite loop, etc.

How many tests do I need?

For the grade? Enough to pass the mutation tests

In life? ... Consider all the ways the function might behave:

- Normal / happy case (expected inputs)
 - `assert 5 == add(2, 3)`
 - `assert 1 == add(2, 3)`
 - `assert 'A' == calculateGrade(96)`
- Invalid inputs
 - `with pytest.raises(ValueError): calculateGrade(-600)`
 - `with pytest.raises(ValueError): add('two', 3)`
 - `with pytest.raises(ValueError): get_area_of_rectangle(-1, 4)`

Edge cases

- Edge cases at the boundaries (almost invalid, but not quite)
 - `assert 0 == get_area_of_rectangle(0, 4)`
 - `assert 0 == divide(0, 1)`

If the function has conditionals, make sure to have test cases for each branch.

Poll: We're testing a function `calculateGrade(score: int)` → `str` that returns a letter grade given a percentage. Which test case is most important to include?

1. `assert 'B+' == calculateGrade(87)`
2. `assert 'F' == calculateGrade(0)`
3. `with pytest.raises(ValueError): calculateGrade(-600)`
4. All of these are equally important



Bill Sempf

@sempf

QA Engineer walks into a bar. Orders a beer. Orders 0 beers. Orders 9999999999 beers. Orders a lizard. Orders -1 beers. Orders a sfdeljknesv.

The CS 2100 Semester Project

Project Guidelines: <https://github.com/neu-pdi/cs2100-public-resources/blob/main/docs/pages/26fa/project.pdf>

There are four types of assessments this semester:

- Weekly assignments (independent question sets)
- Five quizzes (in class on paper)
- In-class exercises (mostly polls)
- **The Semester Project: an open-ended project where you get to choose how to incorporate the required topics**

The CS 2100 Semester Project

Project Guidelines: <https://github.com/neu-pdi/cs2100-public-resources/blob/main/docs/pages/26fa/project.pdf>

Why project? It's a shift from:	To:
"Write this exact essay," but for code	"Practice using this vocabulary word in a sentence"
Entirely autograded work	Grades come from discussing your work with a TA
All students have the same project on their resume	Students get to design their projects

Poll:

- 1. What is your main takeaway from today?**
- 2. What would you like to revisit next time?**